

THE HIDDEN DELIVERY TAX

14 DAYS
mostly waiting.

1.5 days

writing the change

< 1 hour

build + deploy work

~14 days

queues, handoffs, tests

**The bottleneck was not
typing.**

WAIT... THEY SHIPPED HOW OFTEN?

**ONE DEPLOY
EVERY 11.7 SEC**

~23K production deployments per day across
Amazon

Context check

Historical 2011 aggregate—not one app, not a current AWS-wide rate. Many autonomous teams + many services + automation.

AWS STUDENT COMMUNITY DAY · ISLAMABAD

FROM CODE TO CLOUD

Merge one PR. Watch AWS ship the website.

FarhanAshrafDev · DevOps without the corporate yapping

THE MISSION

Make change boring—in a good way.

Fast

How quickly can value reach a user?

Safe

How often does change create failure?

Repeatable

Can the same path run every time?

Recoverable

How quickly can we learn and restore?

THE PROBLEM STATEMENT

“Works on my machine.” Cool. Now what?

ZIP the code and send it in chat

NEXT

Someone remembers the console clicks

NEXT

Production is somehow “different”

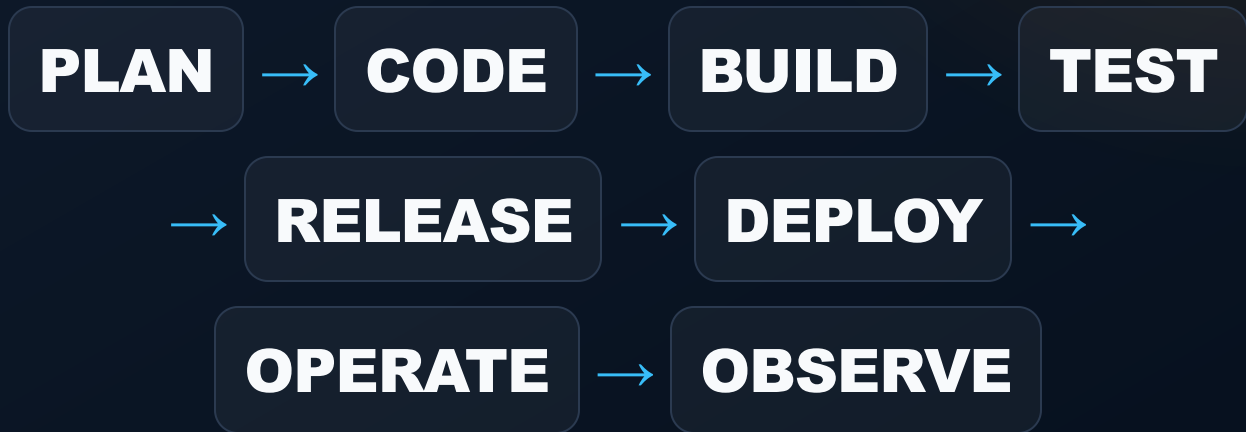
NEXT

Rollback = find yesterday’s ZIP

Memory is not a deployment system.

SOFTWARE DEVELOPMENT LIFE CYCLE

SDLC is a loop, not a finish line.



Observation becomes the next plan.

WHERE DELIVERY GETS STUCK

The queues eat the calendar.

Handoffs

“My part is done.”

Big batches

Too much changes at once.

Late checks

Feedback arrives after
context is gone.

Drift

Every environment has a
secret recipe.

Optimize the **flow**, not one person.

HOW THE MOVEMENT STARTED

2009 changed the conversation.

Velocity 2009

Flickr: “10+ deploys per day” through Dev + Ops cooperation

Oct 2009

Patrick Debois organizes the first DevOpsDays in Ghent

Today

Cloud APIs make environments and delivery paths programmable

DEV + OPS + EVERYONE RESPONSIBLE

DevOps is an operating model.

CULTURE

AUTOMATION

LEAN FLOW

MEASUREMENT

SHARING

OWNERSHIP

Not a tool. Not a job title. Not
“give prod access to
everyone.”

BEFORE VS. AFTER DEVOPS

Same humans. Better system.

Before

- Silos
- Large releases
- Manual checklists
- Slow blame loops

After

- Shared outcomes
- Small changes
- Automated evidence
- Fast learning loops

Speed comes from **confidence.**

THE DEVOPS FEEDBACK LOOP

One change. Five questions.

CODE · what changed?

[NEXT](#)

BUILD · what will we ship?

[NEXT](#)

CHECK · is the evidence good?

[NEXT](#)

DEPLOY · how does it go live?

[NEXT](#)

OBSERVE · did it work?

CI • CONTINUOUS INTEGRATION

Merge small. Check immediately.

SMALL BRANCH + PULL REQUEST

↓ **MERGE OFTEN**

BUILD + TEST + REVIEW EVIDENCE

↓ **FAST FEEDBACK**

MAIN STAYS HEALTHY

CI answers: “Can this change safely join everyone else’s work?”

CD HAS TWO MEANINGS

Ready to release vs. released.

Continuous Delivery

Every passing change is kept releasable; a human may approve production.

Continuous Deployment

Every passing change is automatically released to production.

Today: merge to `main` →
automatic deployment.

X AS CODE

Put repeatable intent in Git.

APP AS CODE

TESTS AS CODE

**PIPELINE AS
CODE**

INFRA AS CODE

CONFIG AS CODE

POLICY AS CODE

DOCS AS CODE

**RUNBOOKS AS
CODE**

Version it · review the diff · automate it · recover from history

INFRASTRUCTURE AS CODE

Stop clicking. Start describing.

ClickOps

- Console memory
- Hidden drift
- Hard to reproduce
- Hard to clean up

CloudFormation

- Blueprint in Git
- Reviewable diff
- Repeatable stack
- Delete together

Intent → **API** → **consistent state**

CLOUD-NATIVE DESIGN VOCABULARY

The 12-factor app in one screen.

01 Codebase

02 Dependencies

03 Config

04 Backing services

05 Build / release / run

06 Processes

07 Port binding

08 Concurrency

09 Disposability

10 Dev/prod parity

11 Logs

12 Admin processes

DO NOT CARGO-CULT THE CHECKLIST

Principles first. Tools second.

One codebase

Git records the source of truth.

Build / release / run

The pipeline separates checked artifacts from runtime.

Parity + logs

Repeat the path; keep visible evidence.

Our static site is not a full SaaS app. It teaches the delivery habits.

SIX AWS BUILDING BLOCKS

Six tools. One job each.

CloudFormation

Create the system

CodeConnections

Trust GitHub once

CodePipeline

Coordinate stages

CodeBuild

Check + package

Amazon S3

Store private files

CloudFront

Serve HTTPS globally

TODAY'S ARCHITECTURE

PR merge to live browser.

GITHUB PR → MERGE TO MAIN

↓ **CODECONNECTIONS**

CODEPIPELINE · SOURCE



CODEBUILD · CHECK + PACKAGE



DEPLOY → PRIVATE S3 → CLOUDFRONT

Commit = record · Pipeline = checklist · AWS = runtime

EVERY STAGE HAS A CONTRACT

No magic. Inputs and outputs.

SOURCE

main commit → source artifact

CHECK

source → validated website artifact + logs

DEPLOY

website artifact → private S3 objects

A failed contract stops the next stage.

ONE STACK + ONE TRUST DECISION

Launch once. Merge forever.

1 · Launch the CloudFormation stack

NEXT

2 · Authorize the pending GitHub connection

NEXT

3 · Merge PRs and watch deployment

Human gate

Choose which repository
AWS may read.

Machine loop

Repeat Source, Check,
Deploy.

LIVE-DEMO MISSION

Merge v2. Watch it move.

- ☒ Show the v1 website
- ☒ Review a three-line PR
- ☒ Merge it into `main`
- ☒ Watch Source → Check → Deploy
- ☒ Refresh and reveal v2

Chat: green pipeline or plot twist?

NO SLIDES FOR THE NEXT 8 MINUTES

PR → MERGE
→ LIVE

Review / Pipeline / Evidence / Website

WHAT JUST HAPPENED?

Follow the evidence trail.

PR review + merge commit

NEXT

Pipeline execution tied to that commit

NEXT

CodeBuild checks + packaged artifact

NEXT

Private S3 deployment + CloudFront response

Traceable beats “trust me, bro.”

PRODUCTION MINDSET

Red can be useful feedback.

Fail early

Stop before users see bad output.

Recover normally

Revert through the same path.

Limit access

Private S3 + scoped IAM roles.

Clean up

Delete the stack; low cost ≠ zero cost.

No AWS key in GitHub. One approved app connection.

**YOUR CHALLENGE: FORK · DEPLOY · BREAK ·
RECOVER · EXPLAIN**

**SMALL.
AUTOMATED.
OBSERVABLE.**



Fsociety-Bulc / AWSSCDWorkshop

One-stack template · PR demo · slides · runbook · cleanup